Data Cleaning & Preprocessing Python Project

1. Project Overview

This project is designed to showcase **an end-to-end, production-grade 17-point data cleaning and feature engineering framework using Python and Pandas.** The focus is on handling extreme real-world data corruption, type mismatches, structural misalignments, missing values, anomalies, and feature transformation to turn chaotic raw data into a pristine, analysis-ready dataset.

---

2. Project Objective

To demonstrate advanced data hygiene and preprocessing skills by systematically resolving complex data errors **such as mixed data types, structural shifting, string-based numeric noise, datetime anomalies, duplicate records, outliers, and automated feature creation** ensuring the dataset is fully optimized for subsequent data workflows.

---

3. Tools & Libraries Used

Python, Pandas, NumPy, Re (integrated within a **Jupyter Notebook** environment

---

4. Project Workflow

The project followed a rigorous methods to transition raw, corrupted data into a production-ready asset:

1. **Inspection & Profiling**
   - Conducted an exhaustive **structural audit using Pandas (.info(), .isnull(), .apply(type))** to isolate high-risk object columns (Age, Order_Date, MRP), detect hidden null patterns, and map out initial data type mismatches.

2. **Targeted Column Remediation & Structural Correction**
   - *Structural Re-alignment:* Resolved cell-shift anomalies where values (such as gender strings) leaked across adjacent columns by shifting records programmatically.

   - *Type Harmonization:* Converted messy mixed-type columns into memory-efficient, nullable extension types (Int64, float64, and datetime64[ns]).

- *Anomaly & Outlier Sanitization:* Purged invalid calendar entries (31-02-2026), future/legacy out-of-bounds dates, negative sign flips (Quantity = -1), and extreme keypunch outliers (Shipping_Cost = 2500).

- *Text & Encoding Repair:* Cleaned whitespace, enforced title casing, and vectorized Latin-1 byte re-encoding to completely fix UTF-8 Mojibake corruption (RenÃ©, Classic T-ShÃ©rt).

3. **Advanced Data Integrity & Relational Auditing**

- Executed relational cross-tabulation audits across the geographic triad (City, State, Pincode) and product hierarchies (Product_ID, SKU) to guarantee 100% relational consistency and zero drift.

- Filtered out system test records (DELETE_ME_STAGING_TEST) and aggregate/forecast summary rows to enforce strict transaction-level granularity.

---

5. What Was Done in This Project ?

This project demonstrates end-to-end mastery of data hygiene by identifying, auditing, and programmatically resolving all **17 industry-standard categories of data quality problems** within a single production-grade dataset.

#	Data Quality Category	Status	Concrete Solution Implemented in Pipeline
1	**Missing / Incomplete Data**	✅ Resolved	Audited null distributions; programmatically imputed missing keys (Order_ID, Customer_ID) and category modes (Quantity).
2	**Duplicate Data**	✅ Resolved	Executed duplicate row checks and unique ID assertions across primary keys (Order_ID, Customer_ID).
3	**Inconsistent Formatting**	✅ Resolved	Standardized text fields using .str.strip(), string collapse regex, and canonical capitalization rules.
4	**Inconsistent Categorical Values**	✅ Resolved	Mapped and consolidated messy variants (MALE, M Male; upi UPI).
5	**Incorrect Data Types**	✅ Resolved	Harmonized datatypes (cast Age and Quantity to nullable extension type Int64, unified financial metrics to float64).

#	Data Quality Category	Status	Concrete Solution Implemented in Pipeline
6	**Date & Time Problems**	✅ Resolved	Parsed mixed datetimes, eliminated out-of-bounds historical dates (2020), corrected impossible calendar dates (31-02-2026), and stripped timezones.
7	**Numerical Data Problems**	✅ Resolved	Corrected negative sign flips (Quantity = -1), impossible zero-value placeholders, and extreme keypunch outliers (Shipping_Cost = 2500).
8	**Text / String Problems**	✅ Resolved	Cleaned typos, invisible spacing, and punctuation artifacts across textual metadata columns.
9	**Structural Problems**	✅ Resolved	Fixed shifted row misalignments (where category attributes leaked into adjacent columns) and reordered 32 columns into logical thematic blocks.
10	**Relationship / Integrity Problems**	✅ Resolved	Verified relational integrity via cross-tabulation audits across the geographic triad (City, State, Pincode) and product hierarchies (Product_ID, SKU).
11	**Logical / Business Rule Errors**	✅ Resolved	Engineered dependent financial aggregates (Discount_Amount, Selling_Price, Item_Subtotal, Tax_Amount, Total_Invoice_Value) following strict retail logic.
12	**Outliers & Anomalies**	✅ Resolved	Identified and sanitized extreme pricing/discount overrides (e.g., fractional discount rate errors like 1.2 and absolute price entries stored as rates).
13	**Unit / Measurement Problems**	✅ Resolved	Harmonized numerical currency and percentage representations to uniform unit standards.
14	**Encoding / Character Problems**	✅ Resolved	Repaired UTF-8 Mojibake corruption artifacts (RenÃ©, Classic T-ShÃ©rt) using vectorised Latin-1 byte re-encoding.
15	**Data Granularity Problems**	✅ Resolved	Ensured strict transaction-level granularity by isolating and dropping aggregate/forecast summary rows.

#	Data Quality Category	Status	Concrete Solution Implemented in Pipeline
16	**Irrelevant / Redundant Data**	✅ Resolved	Filtered and purged system staging records, demo templates, and test check rows (DELETE_ME_STAGING_TEST).
17	**Sensitive / Privacy Issues**	✅ Resolved	Masked and neutralized unverified testing payloads, exposed testing domains (.test), and sanitized privacy leakage suffixes (_CC:FAKE...).

5.1 Advanced Feature Engineering : Generated rich analytical features from cleaned attributes (e.g., customer age groups, tax amounts, selling prices, shipping costs, order time attributes like Is_Weekend, and discount metrics).

5.2 Feature Engineering Code :

1. **'Financial & Revenue Feature Engineering'**

```
# 1. Absolute discount monetary value per unit
clean_data['Discount_Amount'] = (clean_data['MRP'] * clean_data['Discount']).round(2)
# 2. Final selling price per unit after discount
clean_data['Selling_Price'] = (clean_data['MRP'] - clean_data['Discount_Amount']).round(2)
# 3. Subtotal for the line item (Selling Price * Quantity)
clean_data['Item_Subtotal'] = (clean_data['Selling_Price'] * clean_data['Quantity']).round(2)
# 4. Actual tax monetary value applied on the subtotal
clean_data['Tax_Amount'] = (clean_data['Item_Subtotal'] * clean_data['Tax']).round(2)
```

```
# 5. Grand Total (Subtotal + Tax + Shipping Cost)
clean_data['Total_Invoice_Value'] = (clean_data['Item_Subtotal'] + clean_data['Tax_Amount'] + clean_data['Shipping_Cost']).round(2)
```

2. **'Temporal Feature Engineering'**

```
# 1. Extract Month Name and Day Name
clean_data['Order_Month'] = clean_data['Order_Date'].dt.month_name()
clean_data['Order_Day'] = clean_data['Order_Date'].dt.day_name()
# 2. Create a binary flag for weekend orders (1 for weekend, 0 for weekday)
clean_data['Is_Weekend'] = (clean_data['Order_Date'].dt.dayofweek >= 5).astype(int)
```

3. **'Demographic Segmentation'**

```
# 1. Define bins and labels for age cohorts
bins = [0, 25, 40, 60, 100]
labels = ['Young Adult', 'Adult', 'Middle-Aged', 'Senior']
clean_data['Age_Group'] = pd.cut(clean_data['Age'], bins=bins, labels=labels, right=True
```

5.3 <u>Auditing New Columns</u> :

```
new_cols = [
  'Age_Group', 'Discount_Amount', 'Selling_Price', 'Item_Subtotal',
  'Tax_Amount', 'Total_Invoice_Value', 'Order_Month',
  'Order_Day', 'Is_Weekend'
]
display(clean_data[new_cols].info())
```

Output:

```
RangeIndex: 63 entries, 0 to 62
Data columns (total 9 columns):
 #  Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0  Age_Group         57 non-null    category
 1  Discount_Amount    63 non-null    float64
 2  Selling_Price      63 non-null    float64
 3  Item_Subtotal      63 non-null    Float64
 4  Tax_Amount         63 non-null    Float64
 5  Total_Invoice_Value  63 non-null    Float64
```

6 Order_Month 61 non-null object

7 Order_Day 61 non-null object

8 Is_Weekend 63 non-null int32

dtypes: Float64(3), category(1), float64(2), int32(1), object(2

5.4 Data Export to Csv :

filename = "Dataset_cleaned_with_pandas.csv"

clean_data.to_csv(filename, index=False)

print(f"✓ Successfully exported the Dataset_cleaned_with_pandas with {len(clean_data)} rows and {len(clean_data.columns)} columns to '{filename}'!"

5.5 Before vs After [.info()] :

Before

RangeIndex: 72 entries, 0 to 71

Data columns (total 23 columns):

#	Column	Non-Null Count	Dtype
0	Order_ID	71 non-null	object
1	Customer_ID	68 non-null	object
2	Product_ID	70 non-null	object
3	Customer_Name	71 non-null	object
4	Age	65 non-null	object
5	Gender	69 non-null	object
6	Email	70 non-null	object
7	Phone	68 non-null	object
8	City	70 non-null	object
9	State	71 non-null	object
10	Pincode	72 non-null	object
11	Product_Name	72 non-null	object
12	Category	72 non-null	object

13 Brand 72 non-null object

14 SKU 72 non-null object

15 Order_Date 72 non-null object

16 Order_Status 72 non-null object

17 Payment_Method 70 non-null object

18 MRP 72 non-null object

19 Discount 72 non-null float64

20 Tax 70 non-null float64

21 Quantity 70 non-null float64

22 Shipping_Cost 72 non-null int64

dtypes: float64(3), int64(1), object(19)

After

RangeIndex: 63 entries, 0 to 62

Data columns (total 32 columns):

#	Column	Non-Null Count	Dtype
0	Order_ID	63 non-null	object
1	Order_Date	61 non-null	datetime64[ns]
2	Customer_ID	63 non-null	object
3	Customer_Name	63 non-null	object
4	Age	57 non-null	Int64
5	Age_Group	57 non-null	category
6	Gender	63 non-null	object
7	Email	63 non-null	object
8	Phone	63 non-null	object
9	City	63 non-null	object
10	State	63 non-null	object
11	Pincode	63 non-null	object
12	Product_ID	63 non-null	object
13	Product_Name	63 non-null	object
14	Category	63 non-null	object

15 Brand 63 non-null object

16 SKU 63 non-null object

17 MRP 63 non-null int64

18 Discount 63 non-null float64

19 Discount_Amount 63 non-null float64

20 Selling_Price 63 non-null float64

21 Quantity 63 non-null Int64

22 Item_Subtotal 63 non-null float64

23 Tax 63 non-null float64

24 Tax_Amount 63 non-null float64

25 Shipping_Cost 63 non-null float64

26 Total_Invoice_Value 63 non-null float64

27 Payment_Method 63 non-null object

28 Order_Status 63 non-null object

29 Order_Month 61 non-null object

30 Order_Day 61 non-null object

31 Is_Weekend 63 non-null int32

dtypes: Int64(2), category(1), datetime64[ns](1), float64(8), int32(1), int64(1), object(18)

---

6. Business Focus & Use Case

It serves as a high-impact **Data Hygiene Showcase**. It proves the technical capability to take messy, real-world raw data from various enterprise domains and sanitize it into a trustworthy, production-ready format.

---

7. Conclusion

This project demonstrates robust data analytics practices, attention to detail, and proficiency in Pandas. It highlights the ability to conquer the hardest and most tedious part of any data lifecycle: transforming chaotic raw data into pristine working data.
